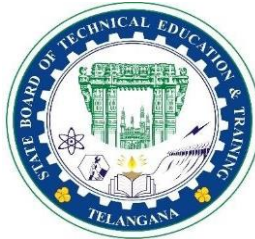


**TRAINING REPORT
ON
Software for Public Fund System among DRDL
Employees**



**Defence Research and Development Laboratory
(DRDL) Kanchanbagh, Hyderabad – 500058
Submitted in partial fulfillment of the degree of
Diploma in Computer Science and Engineering**



**Government Polytechnic for Women Minorities,
Badangpet**

Submitted By:

Gajula Hasini

Doma Pravalika

23097-cs-022

23097-cs-052

Under the Guidance of:

**Shri B.V. HariKrishna Nanda SCIENTIST
'G'Technology Director, DIT, DRDL**

DECLARATION

We Doma Pravalika and Gajula Hasini are here by declare that this report is being submitted partial fulfillment of the requirements of my internship at Defence Research & Development Laboratory (DRDL).

This report is an original work carried out by us during our internship & have been completed under the guidance of **Shri. V. Hari Krishna Nanda, Scientist 'G' & Smt. GMV Lakshmi, TO 'C'.**

This internship was undertaken as a part of the academic curriculum of Government Polytechnic for Women Minorities, Badangpet.

Signature:

Doma Pravalika

Signature:

Gajula Hasini

Signature:

Smt. GMV Lakshmi, TO 'C'

Signature:

B.V HARI Krishna Nanda, Scientist 'G'

Technology Director, DIT, DRDL

Acknowledgement

I wish to express my sincere gratitude to the Defence Research and Development Laboratory (DRDL) for providing a highly professional environment to undertake this work.

I am deeply indebted to **Shri. B.V. Hari Krishna Nanda, SCIENTIST 'G'** for granting me the invaluable opportunity to work on this project and for his continuous guidance. I extend my heartfelt thanks to **Ms. GMV Laxmi, To 'C'** for her unwavering support and technical insights throughout the project duration.

My gratitude also goes to the entire Department team for their cooperative spirit and constant encouragement.

Additionally, I am thankful to the Library and other administrative staff at DRDL for providing seamless access to essential resources and facilities.

Signature:

Doma Pravalika

23097-cs-052

Signature:

Gajula Hasini

23097-cs-022

The Defence Research and Development Laboratory (DRDL) is one of India's most important missile development laboratories under the Defence Research and Development Organisation (DRDO). It is located in Kanchanbagh, Hyderabad and plays a central role in India's missile technology development.



Established in 1961, the laboratory plays a foundational role in India's strategic defense infrastructure and aerospace technologies. Today, the laboratory is globally recognized for designing, developing, and successfully evaluating critical weapon platforms— including the Agni, Prithvi, Akash, and Nag missile families, alongside the supersonic Brahmos cruise missile—thereby ensuring the nation's self-reliance in tactical and strategic missile systems.

HISTORY

The Defence Research and Development Laboratory (DRDL) was established in June 1961 in New Delhi as a small, specialized unit operating within the Defence Science Laboratory. Recognizing the urgent need for vast, isolated testing grounds and dedicated engineering facilities, the Ministry of Defence relocated the laboratory to Hyderabad in 1962. DRDL initially occupied the historic, repurposed Nizam's army barracks in Phoolbagh an area now known as Kanchanbagh. This move provided the physical footprint required to lay down the permanent structural infrastructure for India's indigenous missile design.

Early Experimental Programs (1960s–1970s). During its first two decades, DRDL undertook highly ambitious classified projects that laid the foundational expertise for Indian aerospace engineering. In the 1960s, the laboratory launched Project Indigo, an early attempt to engineer a Surface-to-Air Missile (SAM) that was ultimately halted by strict international technology embargoes. In the 1970s, DRDL transitioned into Project Devil and Project Valiant, which focused on reverse-engineering the Soviet SA-2 liquid-propellant missile and designing a long-range ballistic system.

While the government formally shelved both programs before mass production, they served as critical learning crucibles, training a generation of DRDL scientists in liquid rocket propulsion, telemetry, and advanced fabrication.

The IGMDP Golden Era (1983–2008). The true turning point for the laboratory arrived in 1982 when Dr A.P.J. Abdul Kalam took charge as Director, radically restructuring the laboratory into a unified, mission driven powerhouse. In 1983, the Government of India officially sanctioned the Integrated Guided Missile Development Programme (IGMDP), appointing DRDL as the lead design architecture house.

ABSTARCT

The Public Fund Management System is a web-based application developed to simplify and automate public fund operations within an organization. The system is built using Spring Boot for backend development, React for frontend design, and Oracle Database for database management.

The main objective of this project is to reduce manual paperwork, improve data accuracy, and provide efficient management of fund-related activities such as bill processing, receipt generation, and personnel record maintenance. The application provides a secure login system and user-friendly interface for authorized users to store, update, and retrieve information efficiently.

The system uses REST APIs for communication between frontend and backend, ensuring fast and reliable data processing. Proper validation and database integration help maintain secure and errorfree financial records. The application improves transparency, minimizes human errors, and enhances the overall efficiency of public fund administration. By replacing traditional, fragmented systems with this unified digital solution, the application significantly enhances institutional transparency, minimizes human error, ensures regulatory compliance, and optimizes the overall administrative efficiency of public fund governance.

TABLE OF CONTENT

	CONTENT
1	Certificate
2	Declaration
3	Acknowledgement
4	History
5	Abstract
6	Table of content
7	List of figures
	PART-A
1	Introduction
2	System Architecture
3	Testing and Evaluation
4	Output snapshots
5	Conclusion and Future Enhancement
	PART-B
Chapter-1	INTRODUCTION
1.1	Background

1.2	Objectives of the project
1.3	Problem statement
1.4	Scope of the project
1.5	Tools and Technologies Used
Chapter-2	SYSTEM DESIGN AND IMPLEMENTATION
2.1	System Architecture
2.2	System Design, Database Design
2.3	Module Description
2.4	Implementation
2.4.1	Frontend Implementation
2.4.2	Backend Implementation
2.4.3	Database Implementation
2.4.4	System Integration
2.5	Workflow of the System
Chapter-3	TESTING AND EVALUATION
Chapter-4	OUTPUT
4.1	Output Snapshots
4.2	Challenges Faced and Learning Outcomes

Chapter-5	CONCLUSION & FUTURE ENHANCEMENT
	LIST OF FIGURES
Fig no	Figure Name
2.1.1	System Flow
2.4.1	Cheque Receipt frontend
2.4.2	CDA Bill page frontend
2.4.3	Login Repository Backend
2.4.4	Receipts Service Backend
4.1.1	4.1.1. Receipts Page
4.1.2	4.1.2 Receipts from CDA Page.
4.1.3	4.1.3 PCDA Bill Page
4.1.4	4.1.4 PCDA Bill Page
4.1.5	4.1.5 Cash Receipt Page
4.1.6	4.1.6 Cheque Receipt Page

ORIENTATION OF SOFTWARE APPLICATION AND TOOLS USED AT DRDL

Understanding the software environment

The software environment at DRDL demands secure data handling. The Public Fund Management System operates on a three-tier architecture. The presentation tier displays the user interface inside the browser. The application tier manages business logic on the backend server. The data tier ensures secure persistence in the database. This architecture isolates core computations from direct client access. It guarantees high security and system fault tolerance during operations.

Learning tools

Building this platform requires enterprise-grade software development tools. Developers use Spring Tool Suite and Visual Studio Code IDEs. These environments facilitate backend compilation and frontend script configuration. Apache Maven automates building processes and manages external software dependencies. Oracle SQL Developer assists in schema creation and query execution. Postman serves as the primary tool to test API endpoints.

Learning and Development Tools Spring Tool Suite (STS) / VS Code: Used as primary development IDEs for backend compiling and modular frontend scripting. Apache Maven: Manages backend software lifecycles, automated builds, and external dependency files. Oracle SQL Developer: Used to configure database tables, set integrity constraints, and optimize search indexes. Postman API Client: Used to validate, debug, and perform behavioral verification on REST API endpoints.

Design of software system

The design framework structures the Public Fund Management System cleanly. React.js manages client side components like forms and financial ledgers. Spring Boot backend services handle authentication checks and core workflows. The Oracle Database functions as the

secure relational storage repository. Data transfers between these tiers using lightweight RESTful API contracts. This decoupled design ensures high modularity and scalable system maintenance.

Development of the software system

Software development translates system designs into executable program components. Developers implement secure login modules using distinct username and password inputs. The authentication module runs role-based filters to verify user identities. The frontend uses React hooks to manage dynamic view updates. Axios instances handle communication requests by passing serialized JSON data blocks. Database tables securely store credentials using encrypted hash strings to protect logins.

Testing and integration of software

Rigorous testing and integration verify complete application stability before deployment. Unit testing evaluates standalone backend services and frontend view inputs. Engineers utilize Postman to check RESTful API response status codes. Database integration testing confirms stable connection pooling under heavy transaction loads. End-to-End testing runs complete operational cycles from user login to final ledger updates. This ensures the entire integrated application executes smoothly without errors. If you need any adjustments to the wording, let me know if you would like to change specific technology names or add more details to any section.

Chapter 1

Introduction

1.1 Background [Existing system]

In the existing system, public fund-related transactions are mostly handled through manual or semi-manual processes. Fund allocation, receipt management, bill processing, and approval workflows are often maintained using paper records, spreadsheets, or basic standalone applications. This makes the process time consuming and prone to human errors.

Data consistency and transparency are limited because information is stored in separate files or systems without proper integration. Tracking fund utilization, verifying bills against receipts, and ensuring budget limits are maintained require significant manual effort from administrative staff.

Additionally, approval processes in the existing system are not fully streamlined, leading to delays in processing transactions. There is also a lack of centralized control, making it difficult to monitor all financial activities in real time.

Due to these limitations, there is a need for an automated and centralized system that can efficiently manage receipts, bills, tax calculations, approvals, and fund distribution in a structured manner.

1.2 Objectives of Project

- * To design and develop a centralized Public Fund Management System for handling financial transactions efficiently.
- * To automate receipt, bill, cash withdrawal, and check/DD processing to reduce manual work.
- * To ensure accurate validation of bill amounts against available fund limits.

- * To implement a structured workflow for receipt → bill → final processing stages.
- * To calculate taxes automatically based on selected rates (2%, 10%, or NIL).
- * To maintain consistency and integrity of financial data stored in the database.
- * To provide a secure login system allowing only authorized admin access.
- * To enable better tracking and monitoring of fund utilization and approvals.
- * To reduce errors, delays, and duplication in public fund management processes.
- * To improve transparency and efficiency in financial record handling.

1.3 Problem Statement of Project

The current public fund management process lacks a centralized and automated system for handling financial transactions such as receipts, bill processing, cash withdrawals, and check/DD management. Most of the operations are performed manually or through fragmented systems, which leads to inefficiency, delays, and data inconsistencies.

There is no proper integration between receipt entry, bill generation, tax calculation, and fund distribution, making it difficult to maintain accuracy and real-time tracking of funds. Manual handling of approvals and calculations increases the chances of human errors and reduces transparency in the overall process.

Therefore, there is a need for a secure, structured, and automated system that can efficiently manage end-to-end public fund transactions

with proper validation, controlled access, and accurate financial processing.

1.4 Scope of The Project

- * Managing all fund-related transactions through a centralized web-based system.
- * Handling multiple financial modules such as CDA receipts, PCDA receipts, cash withdrawals, and check/DD processing.
- * Supporting a structured workflow from receipt entry to bill processing and final fund distribution.
- * Ensuring validation of transactions based on available fund limits to avoid over-utilization.
- * Automating tax calculation based on predefined rates (2%, 10%, or NIL).
- * Providing secure login access restricted to authorized admin users only.
- * Maintaining consistent and organized storage of financial data in a database.
- * Enabling tracking and monitoring of all transactions for better transparency and control.
- * Reducing manual intervention, errors, and delays in financial processing.
- * Providing flexibility to extend the system in future for additional financial modules or approval levels.

This system is suitable for organizations that require efficient and reliable management of fund allocation and expenditure with proper validation and workflow control.

1.5 Tools and Technologies Used

Frontend:

React.js

- * Used to develop the user interface of the application
- * Provides component-based architecture for reusable UI components
- * Handles navigation between different modules such as login, receipt, bill, and dashboard pages

Backend:

Spring Boot

- * Used to implement the business logic of the application
- * Handles REST API creation and request processing between frontend and database
- * Built using Maven for dependency management and project build
- * Ensures validation, calculations (tax, fund distribution), and workflow management

Database:

Oracle Database

- * Used to store all transactional and user data securely

- * Manages data related to receipts, bills, cash transactions, and approvals
- * Ensures data integrity and consistency across the system

Development & Supporting Tools:

VS Code – for frontend development (React) and backend code editing

SQL Developer– for managing and executing Oracle database queries

Postman – for testing REST APIs and verifying backend functionality

GitHub– for version control and code sharing with the organization

Chapter2

2.1 System Architecture

The Public Fund Management System follows a 3-tier architecture model, which separates the application into three logical layers:

Presentation Layer, Business Logic Layer, and Data Layer. This helps in better modularity, scalability, and maintenance of the system

2.1.1 Architecture Overview

The system is structured into the following layers:

1. Presentation Layer (Frontend)

- * Built using React.js.
- * Provides user interface for:
 - * Login page
 - * Dashboard
 - * CDA / PCDA / Cash / Check-DD forms
- * Sends user requests to backend using REST APIs
- * Displays processed data received from

2. Business Logic Layer (Backend)

- * Built using Spring Boot (Maven-based)
- * Handles core functionalities such as:
 - * Authentication validation
 - * Receipt and bill processing
 - * Tax calculation (2%, 10%, NIL)
 - * Fund limit validation
 - * Workflow management (receipt → bill → final processing)
- * Communicates between frontend and database using APIs

3. Data Layer (Database)

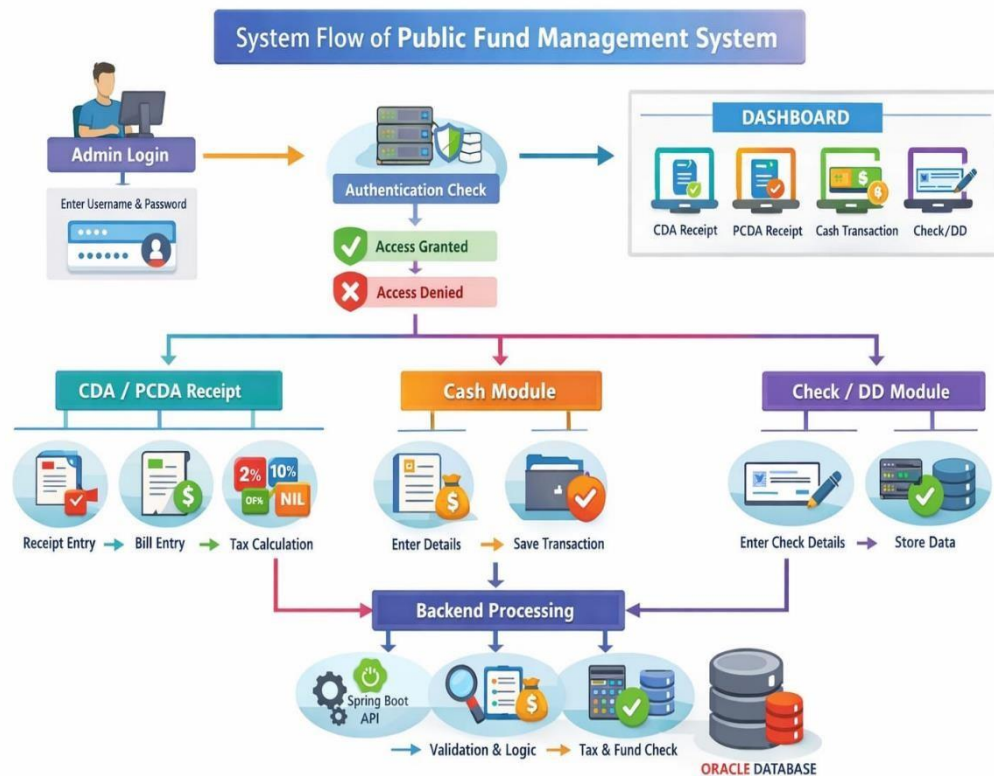
- * Uses Oracle Database

- * Responsible for:
- * Storing user credentials
- * Storing receipt and bill details
- * Storing transaction and approval data
- * Ensures data consistency and integrity

2.1.2 System Flow

System Flow Steps:

1. User accesses the system through the login page
2. Credentials are validated by the Spring Boot backend using database records
3. After successful login, user is redirected to the dashboard
4. User selects a module (CDA Receipt / PCDA Receipt / Cash / Check-DD)
5. Data is entered in the frontend forms (React) 6. Input data is sent to backend through REST APIs
7. Backend performs:
 - * Validation checks
 - * Tax calculation (2%, 10%, NIL)
 - * Fund limit verification
8. Processed data is stored in Oracle database
9. Backend sends response back to frontend
10. UI displays success/failure or processed results to the user



2.1.1 System Flow

2.2 System Design, Database Design

2.2.1 System Design:

The System Design of the Public Fund Management System defines the internal structure and functional behaviour of the application. It explains how different modules interact with each other to perform financial operations such as receipt processing, bill management, tax calculation, and fund validation.

The design focuses on dividing the system into independent functional modules to ensure better control and maintainability.

- Authentication Module
- Dashboard Module
- CDA Receipt Module
- PCDA Receipt Module

- Cash Withdrawal Module
- Check/DD Module
- Transaction Processing Module

2.2.2 Database Design:

The Database Design defines how data is logically structured and stored in the Oracle database. It ensures proper organization of financial records and supports efficient retrieval and processing of transaction data.

Data Organization Approach

The database is structured to store data based on functional grouping of modules rather than a single table structure. Each type of transaction is stored separately but linked logically through identifiers.

Data Categories

1. User Data

- * Stores authentication details of admin users
- * Used for login validation

2. Receipt Data

- * Stores CDA and PCDA receipt information
- * Acts as the starting point of transaction flow

3. Bill Data

- * Stores bill-related financial information
- * Includes amount, purpose, and bill metadata

4. Final Transaction Data

- * Stores processed and validated records
- * Includes tax applied and final distributed amounts

5. Cash/Check/DD Data

- * Stores standalone transaction records
- * Includes reference, purpose, and type information

Database Design Characteristics

- * Data is logically grouped based on transaction type
- * Maintains consistency across receipt, bill, and final records
- * Uses unique identifiers for tracking transactions
- * Supports fast retrieval for reporting and verification
- * Ensures data integrity through structured storage
- * Avoids redundancy by separating functional data types

Tables and their Attributes:

1. R_BILL_ONE

Attributes:

CONTNO

BILLNO

BILLDATE

TYPE

PURPOSE

BILLAMOUNT

BALANCE

RVNO

ALERTFLAG

2. R_DETAILS

Attributes:

RVNO

PERSNO

AMOUNT

SNO

BALANCE

ALERTFLAG

CASHBOOK_AMOUNT

TAXFLAG

3. RECEIPTS_ONE

Attributes:

CONTNO

DVNO_REFNO

RVNO

CHEQUE_RECEIPT_NO

AMOUNT
TRCD
TRCD_DESC
REMARKS
BALANCE
TR_DATE
CHEQUE_RECEIPT_DATE
R_PURPOSE
CHEQUE_CASH
FROM_ADDRESS
R_TYPE
ALERTFLAG
REC_NO
BANK_CREDIT_DATE

4. TRCD

Attributes:

TRCD
DESCRIPTION

5. DRDL_PERSONNEL

Attributes:

PERSNO
NAME

DESIG
DIVISION
DEPARTMENT
IDNO
STATUS
F_CDES_TYPE
BANK_NAME
BANK_ACCNO
BANK_BRANCH
IFSCNEFTCODE
MICRNO
ADDRESS1
CITY
DISTRICT
STATE
PINNO
ADDRESS2
PANNO

6. OTHER_PERSONNEL

Attributes:

PERSNO
NAME
DESIG

DIVISION

DEPARTMENT

IDNO

STATUS

F_CDES_TYPE

BANK_NAME

BANK_ACCNO

BANK_BRANCH

IFSCNEFTCODE

MICRNO

ADDRESS1

CITY

DISTRICT

STATE

PINNO

ADDRESS2

PANNO

7. PF_LOGIN

Attributes:

USERNAME

PASSWORD1

8. PURPOSE

Attributes:

SNO

PURPOSE

TYPE

2.3 Module Description

The Public Fund Management System is divided into multiple functional modules to handle different financial operations in a structured and efficient manner. Each module is responsible for a specific part of the fund management process, ensuring proper validation, processing, and storage of data.

2.3.1 Authentication Module

The Authentication Module is responsible for controlling access to the system.

Functionality:

- * Provides login interface for the admin use
- * Validates user credentials against stored database records
- * Grants access only if credentials are correct
- * Restricts unauthorized access to the system

Importance:

- * Ensures system security
- * Prevents unauthorized financial operations

2.3.2 Dashboard Module

The Dashboard Module acts as the central navigation point after successful login.

Functionality:

- * Displays available financial modules:
- * CDA Receipt
- * PCDA Receipt
- * Cash Withdrawal
- * Check/DD
- * Allows admin to select required module
- * Provides navigation between different transaction types

Importance:

- * Acts as control center of the system
- * Provides structured access to all operations

2.3.3 CDA Receipt Module

This is a multi-step financial processing module.

Functionality:

- * Step 1: Receipt Entry
- * Admin enters receipt number, date, and related details
- * Data is stored and passed to next stage
- * Step 2: Bill Entry
- * Admin enters bill details such as amount, purpose, bill number, and date
- * Bill is linked with receipt data
- * Step 3: Final Processing
- * System performs validation of bill amount

- * Applies tax calculation (2%, 10%, NIL)
- * Distributes amount based on rules
- * Final transaction is generated

Importance:

- * Ensures structured financial processing
- * Maintains proper linkage between receipt and bill

2.3.4 PCDA Receipt Module

The PCDA module functions similarly to the CDA module.

Functionality:

- * Follows same 3-step process:
- * Receipt Entry
- * Bill Entry
- * Final Processing
- * Only classification/type differs from CDA module
- * Stores and processes data separately for PCDA transactions

Importance:

- * Provides separate handling for different fund categories
- * Maintains organized financial classification

2.3.6 Check/DD Module

This module handles cheque and demand draft transactions.

Functionality:

Single interface similar to Cash module

- * Admin enters transaction details

- * System validates and stores data
- * Processes Check/DD financial records

Importance:

- * Manages non-cash financial transactions
- * Ensures proper record keeping of cheque-based payments

2.3.7 Transaction Processing Module (Core Logic Module)

This module represents the internal processing logic of the system.

Functionality:

- * Validates all financial inputs
- * Ensures bill amount does not exceed available funds
- * Applies tax rules (2%, 10%, NIL)
- * Controls flow from receipt → bill → final stage
- * Ensures data consistency before storage

Importance:

- * Acts as the core processing engine
- * Ensures financial accuracy and validation

2.4 Implementation

The implementation of the Public Fund Management System is done using a 3-tier architecture approach, where the frontend, backend, and database are developed and integrated separately. The system is implemented to handle financial transactions such as receipts, bills, cash withdrawals, and check/DD processing in a structured and automated manner.

The implementation focuses on:

- * Secure user authentication

- * Modular transaction processing
- * Data validation and tax calculation
- * Seamless communication between frontend, backend, and database

2.4.1 Frontend Implementation (React.js)

The frontend is developed using React.js to provide an interactive and user-friendly interface for the admin.

Key Implementations

- * Login page for authentication
- * Dashboard with module navigation

Forms for:

- * CDA Receipt
- * PCDA Receipt
- * Cash Withdrawal
- * Check/DD
- * Form validation before sending data
- * API integration with backend using REST calls

Key Features

- * Component-based UI design
- * Controlled form inputs
- * Navigation between modules
- * Real-time response display from backend

2.4.2 Backend Implementation (Spring Boot)

The backend is developed using Spring Boot with Maven, which handles all business logic, validations, and API processing.

Key Implementations

REST API development for all modules

- * Authentication validation using database records
- * Receipt and bill processing logic
- * Tax calculation logic:
 - * 2%
 - * 10%
 - * NIL
- * Fund validation (bill amount vs available funds)
- * Data transfer between frontend and database

Key Features

- * Controller-Service-Repository structure
- * Exception handling for invalid inputs
- * JSON-based request/response handling
- * Modular API design for each function

2.4.3 Database Implementation (Oracle Database)

Oracle Database is used to store and manage all financial and user-related data securely.

Key Implementations

- * Creation of tables for:
 - * User credentials
 - * Receipt data
 - * Bill data

- * Transaction records
- * Cash and Check/DD entries
- * Primary key and foreign key relationships
- * Auto-generated identifiers for tracking records
- * Default values for specific fields (e.g., MIS type, alert flag = N)

Key Features

- * Structured relational database design
- * Data integrity using constraints
- * Efficient query handling
- * Secure storage of financial data

2.4.4 System Integration Implementation

System integration ensures communication between frontend, backend, and database.

Key Implementations

- * React frontend communicates with Spring Boot via REST APIs
- * Spring Boot processes requests and interacts with Oracle DB
- * JSON format used for data exchange
- * End-to-end flow:
- * UI → API → Database → API → UI

Key Features

- * Real-time data transfer
- * Stateless communication using REST

* Modular integration between components

React UI → Spring Boot API → Oracle DB

← Response ←

CODE SINNIPTS

```
import React, { useState } from "react";
import "../ChequeDdReceipt.css";
import { useNavigate } from "react-router-dom";
import axios from "axios";

const ChequeDdReceipt = () => {

  const navigate = useNavigate();

  const [receipt, setReceipt] = useState({
    chequeDdNo: "",
    chequeDdDate: "",
    referenceNo: "",
    purpose: "",
    fromWhom: "",
    amount: "",
    bankCreditDate: "",
  });

  const handleChange = (e) => {

    setReceipt({
      ...receipt,
      [e.target.name]: e.target.value,
    });
  };
};
```

2.4.1 Cheque Receipt frontend

```
body {
  background-color: #e6f2ff;
  font-family: Arial, Helvetica, sans-serif;
  font-size: 13px; /* increased by 1 */}

/* Full Screen Container */

.container {
  position: relative;
  width: 98%;
  margin: 10px auto;
  background: #f7fbff;
  padding: 15px 18px;
  border-radius: 6px;
  border: 1px solid #b7d3f2;
}
```

2.4.2 CDA Bill Page Frontend

```
package com.pf.login.repository;

import com.pf.login.entity.Login;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;

public interface LoginRepository extends JpaRepository<Login, Long> {

    Optional<Login> findByUsername(String username);
}
```

2.4.3 Login Repository Backend

```

package com.pf.login.service;

import com.pf.login.dto.ReceiptRequestDto;
import com.pf.login.dto.ReceiptBillDto;
import com.pf.login.entity.ReceiptOne;
import com.pf.login.entity.Trcd;
import com.pf.login.repository.ReceiptRepository;
import com.pf.login.repository.TrcdRepository;

import org.springframework.stereotype.Service;

@Service
public class ReceiptService {

    private final ReceiptRepository receiptRepository;
    private final TrcdRepository trcdRepository;

    public ReceiptService(ReceiptRepository receiptRepository,
        TrcdRepository trcdRepository) {
        this.receiptRepository = receiptRepository;
        this.trcdRepository = trcdRepository;
    }
}

```

2.4.4 Receipt Service Backend.

2.5 WORKFLOW OF THE SYSTEM

The workflow of the Public Fund Management System describes the complete flow of operations starting from user login to final transaction storage in the database.

The system handles four main modules: CDA Receipt, PCDA Receipt, Cash, and Check/DD

Complete System Workflow (Step-by-Step)

1. The admin accesses the application through the login page.
2. Admin enters valid username and password.

3. The system validates credentials using stored database records.
4. If authentication is successful, the admin is redirected to the dashboard.
5. The dashboard displays available modules:
 - * CDA Receipt
 - * PCDA Receipt
 - * Cash
 - * Check/DD
6. Admin selects any one module based on the required transaction type.
7. If CDA or PCDA module is selected:
 - * Admin enters receipt details (receipt number, date, etc.).
 - * System stores receipt information in the database.
 - * Admin proceeds to bill entry screen.
 - * Bill details such as bill number, date, amount, and purpose are entered.
 - * System validates bill amount against available fund limit.
 - * If valid, system proceeds to final processing stage.
 - * Admin selects tax option (2%, 10%, or NIL).
 - * System calculates tax and performs final fund distribution.
 - * Final transaction record is generated and stored in the database.
8. If Cash module is selected:
 - * Admin enters required details such as reference number, receipt number, purpose, and from address.
 - * System assigns default values like type = MIS and alert flag = N.

- * Data is validated and stored directly in the database as a transaction record.

9. If Check/DD module is selected:

- * Admin enters cheque/DD-related transaction details.
- * System validates the input data.
- * Valid data is stored in the database as a financial record.

10. After each transaction:

- * Backend processes all requests using Spring Boot APIs.
- * System performs required validations and business logic operations.
- * Processed data is sent back as a response to the frontend.
- * Transaction result is displayed to the admin.

11. All finalized data is stored in the Oracle database, ensuring proper record management of:

- * Receipts
- * Bills
- * Cash transactions
- * Check/DD transactions
- * Final processed records

12. The admin can continue new transactions or return to the dashboard for further operations.

Chapter3 Testing and Evaluation

3.1 Testing:

Testing and evaluation are required to ensure that the developed system works correctly and meets all the specified requirements before it is used in real conditions.

The purpose of testing and evaluation in the Public Fund Management System is to ensure that the developed application works correctly, reliably, and securely according to the required specifications.

Testing is performed to verify that all modules such as login, receipt processing, bill handling, cash transactions, and check/DD operations function properly without errors. Types of Testing Used

3.1.1 Unit Testing

- * Each module (login, receipt, bill, etc.) is tested individually
- * Ensures each function works correctly

3.1.2 Integration Testing

- * Checks communication between frontend, backend, and database
- * Ensures data flows correctly through APIs

3.1.3 Functional Testing

- * Verifies that all system features work according to requirements
- * Example: tax calculation, fund validation

3.1.4. System Testing

- * Entire application is tested as a complete system

- * Ensures end-to-end workflow works properly

3.2 Evaluation

The system is evaluated based on:

- * Accuracy of financial calculations
- * Correctness of data storage and retrieval
- * Response time between frontend and backend
- * Security of login system
- * Proper handling of valid and invalid inputs

The evaluation confirms that the system meets all functional requirements of a Public Fund Management System.

3.3 Commands Used for Execution

Since your project uses React + Spring Boot + Oracle DB, the execution commands are as follows:

3.3.1. Frontend (React) Execution

npm install

Install dependencies (first time only) npm start

Starts the React development server

Runs on: <http://localhost:3000>

3.3.2 Backend (Spring Boot) Execution

Using Maven: mvn

clean install

Next Builds the project and downloads dependencies mvn spring-boot:

run

Starts Spring Boot application

Backend runs on: http://localhost:8080

3.3.3. Database (Oracle SQL Developer)

No command execution like terminal is used here.

Instead:

- * Open SQL Developer

- * Connect to Oracle database
- * Run SQL scripts like:

sql

SELECT * FROM table_name;

sql

INSERT INTO table_name VALUES (...);

3.3.4 API Testing (Postman)

- * No command required

- * Use endpoints like:

- * GET / POST / PUT / DELETE

Chapter-4

4.1 OUTPUT SNAPSHOTS

RECEIPTS

From CDA, Hyderabad

From PCDA, DELHI

Cash

Withdrawal

Cheque / DD

Direct From SBI

4.1.1. Receipts Page

PUBLIC FUND
RECEIPT FROM CDA

CDA DV NO :

202605

CDA CHEQUE NO :

CDA CHEQUE DATE :

dd - mm - yyyy

CHEQUE SLIP TOTAL AMOUNT :

BANK CREDIT DATE :

dd - mm - yyyy

SAVE RECEIPT

4.1.2 Receipts From CDA Page.

PUBLIC FUND

Menu

Receipt Details

REC NO : 2026050006

PCDA DVNO : 202605100

PCDA CHEQUE NO : 1009

PCDA CHEQUE DATE : 2026-05-26

BANK CREDIT DATE : 2026-05-26

CHEQUE SLIP TOTAL AMOUNT : 1000

Bill Details

TYPE

PURPOSE

BILL NO

BILL DATE

BILL AMOUNT

Select

Select

dd - mm - yyyy

SAVE BILL

+ Add Another Receipt

4.1.3 PCDA Bill Page

PUBLIC FUND

Menu

Receipt Details

REC NO : 2026050006

PCDA DVNO : 202605100

PCDA CHEQUE NO : 1009

PCDA CHEQUE DATE : 2026-05-26

BANK CREDIT DATE : 2026-05-26

CHEQUE SLIP TOTAL AMOUNT : 1000

Bill Details

RVNO : 2026050021

BILL NO : 127

BILL DATE : 2026-05-26

TAX % : MIS

PURPOSE : m3

BILL AMOUNT : 100

Add Persons

ID No	PER No	Name	Amount	Cashbook Amount	Tax Percentage
202	2002	DIVYA SINGH	10	10	10

CGST 0.00

0.00

SGST 0.00

0.00

Base Amount 10

Net Amount 9.00

TDS Amount 1.00

SAVE

DRDL Employee Details

Other Persons Details

Search by ID / PER NO / Name

2001 - ANIL KUMAR - 201

2002 - DIVYA SINGH - 202

Search by ID / PER NO / Name

1001 - RAVI KUMAR - 101

1002 - SNEHA REDDY - 102

Add Another Bill

Add another Receipt

4.1.4 Billing Page

PUBLIC FUND

CASH RECEIPT

RECEIPT NO :

REFERENCE NO :

FROM ADDRESS :

PURPOSE :

CHEQUE RECEIPT DATE :

AMOUNT :

dd - mm - yyyy

SAVE

4.1.5 Cash Receipt Page

The image shows a web form titled "PUBLIC FUND" with the subtitle "Cheque / DD RECEIPT". The form is set against a light blue background. It contains several input fields and a "SAVE" button. The fields are arranged in two columns. The left column includes "CHEQUE / DD NO :", "REFERENCE NO / LETTER NO :", "FROM WHOM :", and "BANK CREDIT DATE:". The right column includes "CHEQUE / DD DATE :", "PURPOSE :", and "AMOUNT :". The date fields are pre-filled with "dd-mm-yyyy" and have a calendar icon. The "SAVE" button is centered at the bottom of the form.

PUBLIC FUND			
Cheque / DD RECEIPT			
CHEQUE / DD NO :		CHEQUE / DD DATE :	dd-mm-yyyy
REFERENCE NO / LETTER NO :		PURPOSE :	
FROM WHOM :		AMOUNT :	
BANK CREDIT DATE:	dd-mm-yyyy		
SAVE			

4.1.6 Cheque Receipt Page

4.2 Challenges Faced:

During the development of the Public Fund Management System, several technical and implementation challenges were encountered:

*Integration of Frontend and Backend:

Establishing proper communication between React (frontend) and Spring Boot (backend) using REST APIs required careful handling of request and response formats.

*Data Flow Management Across Modules:

Managing data transfer between multiple steps (Receipt → Bill → Final Processing) was complex and required proper state handling.

*Validation of Financial Rules:

Implementing constraints such as bill amount checking against available funds and preventing invalid transactions required accurate backend logic.

***Tax Calculation Logic:**

Handling multiple tax options (2%, 10%, NIL) and applying them correctly during final processing required precise implementation.

***Database Consistency:**

Ensuring proper storage and retrieval of related transaction data across multiple operations in Oracle Database was challenging.

***Debugging and API Testing:**

Identifying errors in API calls and resolving issues during Postman testing required multiple iterations.

4.3 Learning Outcomes:

The development of this project provided strong practical knowledge and technical learning in full-stack application development:

- * Gained hands-on experience in React.js frontend development
- * Learned backend development using Spring Boot and REST APIs
- * Understood database integration using Oracle SQL Developer
- * Improved understanding of 3-tier architecture system
- * Learned how to implement real-time validation and business logic
- * Gained experience in API testing using Postman
- * Developed skills in problem-solving and debugging full-stack applications
- * Understood how financial systems handle work flow based transaction processing
- * Learned version control and project management using GitHub

Chapter 5

Conclusion and Future Enhancements:

Conclusion:

The Public Fund Management System was successfully designed and developed as a web-based application to manage financial transactions in an organized and efficient manner. The system integrates multiple modules such as CDA Receipt, PCDA Receipt, Cash, and Check/DD, enabling structured handling of fund-related operations.

The application ensures a complete workflow starting from user authentication, module selection, data entry, validation, tax calculation, and final storage in the database. The use of a 3-tier architecture (React, Spring Boot, Oracle Database) provides a clear separation of responsibilities, making the system modular, scalable, and easy to maintain.

This system reduces manual effort involved in fund management and minimizes errors in financial processing by implementing proper validation rules and automated calculations. It also improves transparency and accuracy in handling public fund transactions.

Overall, the project successfully achieves its objective of building a secure, efficient, and reliable fund management system with proper workflow control and data integrity.

6.FUTURE ENHANCEMENTS

The Public Fund Management System can be further improved by adding advanced features such as multilevel approval workflows, role-based access control for different users, and real-time notification alerts for transactions. In future, the system can also be enhanced by integrating analytics and reporting dashboards to provide better insights into fund utilization. Additionally, cloud deployment and mobile application support can be added to improve accessibility and scalability of the system.

